

PZ-VL53L0X 激光测距模块开发手册

本手册将教大家如何在普中 F407 最小系统&天马&麒麟开发板上使用 PZ-VL53L0X 激光测距模块。本章分为如下几部分内容：

- 1 PZ-VL53L0X 模块简介
- 2 硬件设计
- 3 软件设计
- 4 实验现象

1 PZ-VL53L0X 模块简介

1.1 特性参数

PZ-VL53L0X 是深圳普中科技推出的一款高性能激光测距模块。该模块采用 ST 公司的 VL53L0X 芯片作为核心，该芯片内部集成了激光发射器和 SPAD 红外接收器，采用了第二代 FlightSense™ 技术，通过接收器所接收到的光子时间来计算距离，最远测量距离可达两米，非常适合中短距离测量的应用。

PZ-VL53L0X 模块具有：体积小、测量精度高、多测量工作模式、支持 IIC 从机地址设置和中断、兼容 3.3V/5V 系统、使用方便等特点。

PZ-VL53L0X 模块各项参数如下图所示：

项目	说明
接口特性	3.3V/5V
通讯接口	IIC 接口
通讯速率	400Khz (Max)
测量精度	±3%
测量范围	3cm~200cm ^①
响应频率	20ms (Max)
工作温度	-20℃~70℃
存储温度	-40℃~85℃
模块尺寸	11.4mm*18.8mm
电源电压	3.3V/5V
I/O 口电平 ^②	3.3V LVTTTL
功耗	12~20ma

注：①：vl53l0x 传感器测量盲区有 3~4cm。②：模块 I/O 电压是 2.8V，不过我们做了 3.3V\5V 兼容性处理（串 120R 电阻）所以可以直接连接 3.3V 和 5V 的 MCU 使用（5V MCU 必须可以识别 2.8V 为高电平才可以）。

1.2 使用说明

1.2.1 模块引脚说明

PZ-VL53L0X 激光测距传感器模块通过 2*4 的排针（2.54mm 间距）同外部连接，模块可以与天马&麒麟开发板直接对接（插 WIFI 接口），而 F407 最小系统

序号	名称	说明
1	VCC	3.3V/5V电源输入
2	IIC_SCL	IIC 通信时钟线
3	VL_XSH	片选使能
4	NC	
5	VL_INT	中断输出引脚
6	NC	
7	IIC_SDA	IIC 通信数据线
8	GND	地线

模块通过 IIC 接口与外部进行通信，上电时，默认的 IIC 从机地址为：0X52，地址的修改需要软件代码进行实现。关于模块从机地址修改后面 API 使用说明小节会介绍。

1.2.2 VL53L0X 简介

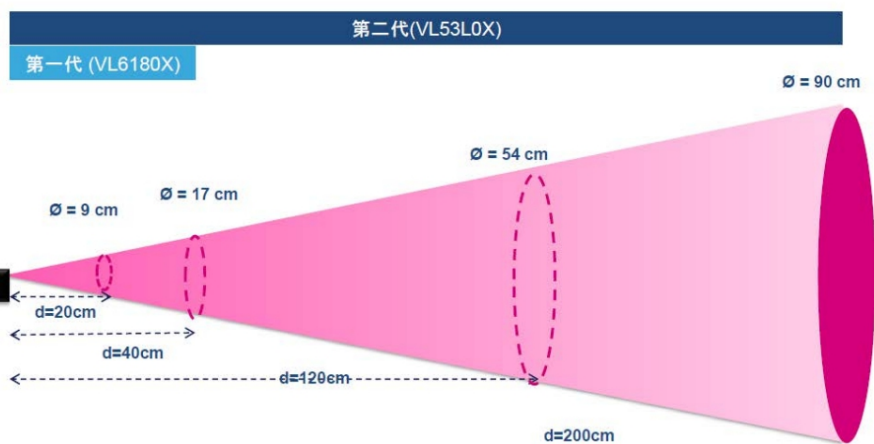
VL53L0X 是 ST 公司推出的新一代 ToF 激光测距传感器，采用了第二代 FlightSense™ 技术，利用飞行时间（ToF）原理，通过光子的飞行来回时间与光速的计算，实现测距应用。比上一代 VL6180x，新的器件将飞行时间测距长度扩展至 2 米，测量速度更快，能效更高。除此之外，为使集成过程更加快捷方便，ST 公司为此也提供了 VL53L0X 软件 API（应用编程接口）以及完整的技术文档，通过主 IIC 接口，向应用端输出测距的数据，大大降低了开发难度。

VL53L0X 的特点包括：

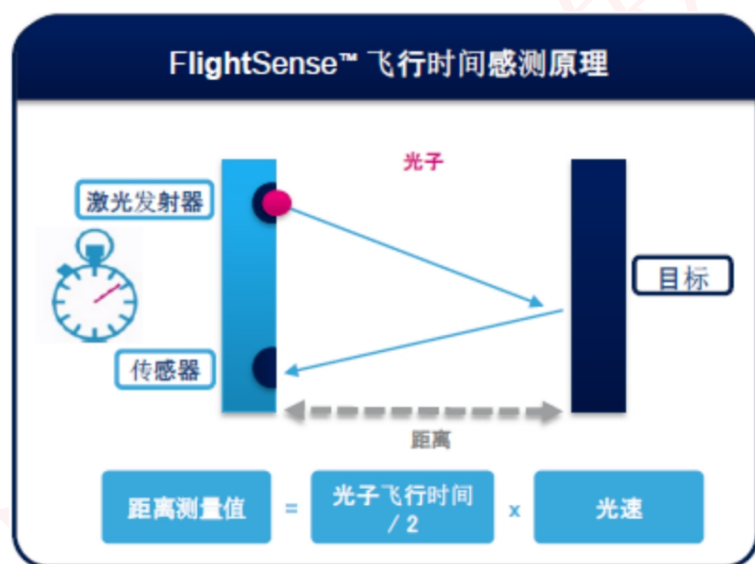
- ①使用 940nm 无红光闪烁激光器，该频段的激光为不可见光，且不危害人眼。
- ②系统视野角度(FOV)可达 25 度，传感器的感测有效工作直径扩展到 90 厘米。
- ③采用脉冲式测距技术，避免相位式测距检测峰值的误差，利用了相位式检测中除波峰以外的光子。
- ④多种精度测量和工作模式的选择。
- ⑤测距距离能扩至到 2 米。
- ⑥正常工作模式下功耗仅 20mW，待机功耗只有 5uA。
- ⑦高达 400Khz 的 IIC 通信接口。
- ⑧超小的封装尺寸：2.4mm × 4.4mm × 1mm。

VL53L0X 传感器系统视野如下图所示：

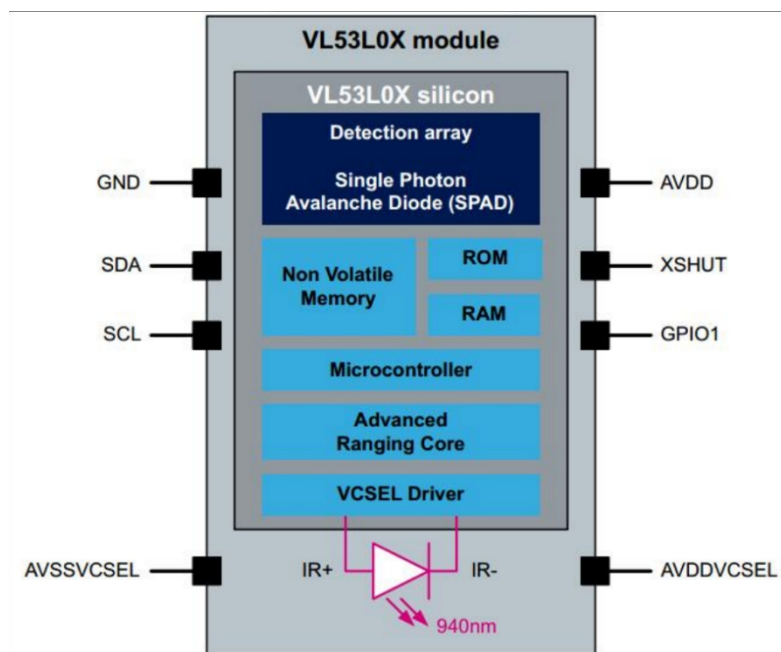
系统视野 (FOV) : 25° (激光发射器 + 接收器)



VL53L0X 传感器的感测原理如下图所示:



VL53L0X 传感器的内部框图, 如下图所示:



从图中可以看到，VL53L0X 集成了人眼不可见的 940nm VCSEL 发射器（垂直腔面发射激光器）。此激光器不会对眼睛造成任何伤害，完全满足针对 1 类激光设备的最新标准（IEC 60825-1:2014-第 3 版）。此外，VL53L0X 还配有内置物理红外滤光片，可增大测量距离、增加对环境光的抗扰度，以及对玻璃罩光学串扰的抗扰度。反射回程的 IR 光通过高灵敏度的领先 SPAD（单光子雪崩二极管）阵列进行测量。

SCL 和 SDA 是连接 MCU 的 IIC 接口，MCU 通过这个 IIC 接口来控制 VL53L0X。XSHUT 为芯片的片选引脚，用于 MCU 使能或复位传感器。而 GPIO1 引脚为中断输出引脚，内部开漏输出，使用时需外部上拉电阻，该引脚可帮助主机控制器中断时序关键型应用，或者在应用无需快速新测距任务时，用作轮询使用。

1.2.3 VL53L0X 工作模式

VL53L0X 传感器提供了 3 种测量模式，Single ranging（单次测量）、Continuous ranging（连续测量）、以及 Timed ranging（定时测量），下面我们简单介绍下这几种模式：

(1) **Single ranging（单次测量）**，在该模式下只触发执行一次测距测量，测量结束后，VL53L0X 传感器会返回待机状态，等待下一次触发。

(2) **Continuous ranging（连续测量）**，在该模式下会以连续的方式执行

测距测量。一旦测量结束，下一次测量就会立即启动，用户必须停止测距才能返回到待机状态，最后的一次测量在停止前完成。

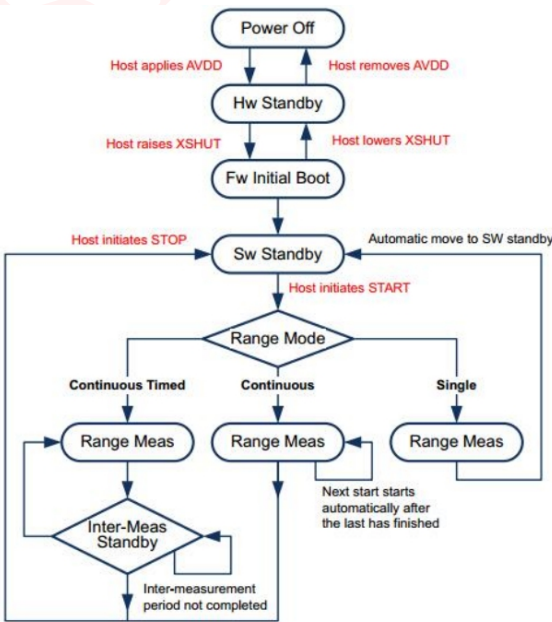
(3) **Timed ranging (定时测量)**，在该模式下会以连续的方式执行测距测量。测量结束后，在用户定义的延迟时间之后会启动下一次测量。用户必须停止测距才能返回到待机状态，最后的一次测量在停止前完成。

根据以上的测量模式，ST 官方提供了 4 种不同的精度模式，如下图所示：

精度模式	测量时间预算范围 (ms)	测距性能 (m)	典型应用
默认	30	1.2	标准
高精度	200	1.2 精度 $\leq\pm 3\%$	精确测量
长距离	33	2	长距离，只适用于黑暗条件（无红外线）
高速	20	1.2 精度 $\pm 5\%$	高速，精度不优先

从图中可以看到，针对不同的精度模式，测量时间也是有所区别的，测量时间最快为高速模式，只需 20ms 内就可以采样一次，但精度确存在有 $\pm 5\%$ 的误差。而在长距离精度模式下，测距距离能达到 2m，测量时间在 33ms 内，但测量时需在黑暗条件（无红外线）的环境下。所以在实际的应用中，需根据当前的要求去选择合适的精度模式，以达到最佳的测量效果。关于 VL53L0X 精度的详细介绍，请看 VL53L0X 芯片手册文档（\3—芯片数据手册\VL53L0X.pdf）5.3 章节。

VL53L0X 简易的工作流程图，如下图所示：



针对测量数据的获取，VL53L0X 提供了轮询和中断两种工作方式，可根据实

际情况进行选择。对于在 VL53L0X 实际测量时，可能会出现距离偏差，官方对此也有对 VL53L0X 传感器校准有一个详细的说明。关于测量数据的获取工作方式和校准的具体说明，请看 VL53L0X 芯片手册文档（VL53L0X.pdf）2.7 和 2.3 章节，在这里我们就不做讲解了。

关于 VL53L0X 的介绍，我们就介绍到这。VL53L0X 的详细资料介绍，请参考“\3--芯片数据手册\VL53L0X.pdf”。

1.2.4 VL53L0X API 使用介绍

经过前面小节介绍，了解到 VL53L0X 工作流程，要对芯片进行使用，ST 并没有直接提供 VL53L0X 寄存器手册，而是提供 VL53L0X 软件 API（应用编程接口）以及完整的技术文档和例程源码，例程源码是 ST 官方的 X-NUCLEO-53L0A1 扩展板基于 STM32F401RE 和 STM32L476RG Nucleo 开发板进行开发的，我们需要将其移植一下才可以用到，官方例程驱动在“\4--参考资料\en.X-CUBE-53L0A1.zip”，en.X-CUBE-53L0A1.zip 就是官方的例程驱动，代码比较多，不过官方提供了 VL53L0X 软件 API（应用编程接口）文档供大家学习：VL53L0X_API_v1.0.2.4823_externalx.chm，这个文件在 API 资料文件夹里面，大家可以阅读这个文件，来熟悉 VL53L0X 驱动库的使用。

官方 VL53L0X 驱动库的移植，需要实现 VL53L0X_WrByte、VL53L0X_WrWord、VL53L0X_WrDWord、VL53L0X_RdByte、VL53L0X_RdWord、VL53L0X_RdDWord、VL53L0X_WriteMulti、VL53L0X_ReadMulti、VL53L0X_UpdateByte、VL53L0X_PollingDelay 等底层驱动函数。具体细节，我们就不详细介绍了，移植后的驱动代码，可以在：模块资料→程序源码→模块实验→APP→v153l0x 文件夹下找到，其中 core 文件，是 API 的功能应用函数。platform 文件，是我们需要实现的底层驱动函数。而 demo 文件，则是模块例程测试实验驱动。各文件夹文件如下图所示：

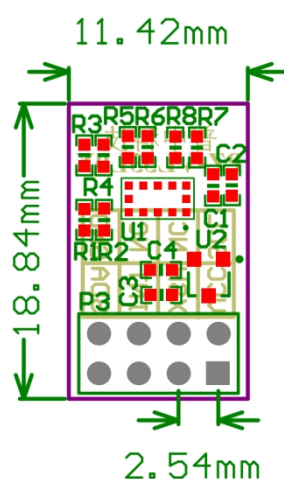
vl53l0x > core	> vl53l0x > platform	> vl53l0x > demo
名称	名称	名称
 vl53l0x_api.c	 vl53l0x_i2c.c	 vl53l0x.c
 vl53l0x_api.h	 vl53l0x_i2c.h	 vl53l0x.h
 vl53l0x_api_calibration.c	 vl53l0x_platform.c	 vl53l0x_cali.c
 vl53l0x_api_calibration.h	 vl53l0x_platform.h	 vl53l0x_cali.h
 vl53l0x_api_core.c	 vl53l0x_platform_log.h	 vl53l0x_gen.c
 vl53l0x_api_core.h	 vl53l0x_types.h	 vl53l0x_gen.h
 vl53l0x_api_ranging.c		 vl53l0x_it.c
 vl53l0x_api_ranging.h		 vl53l0x_it.h
 vl53l0x_api_strings.c		
 vl53l0x_api_strings.h		
 vl53l0x_def.h		
 vl53l0x_device.h		
 vl53l0x_interrupt_threshold_settings.h		
 vl53l0x_tuning.h		

1.2.5 模块使用注意事项

模块属于光学器件，保存时需要注意防尘防潮。在使用时，需保持传感器表面的清洁度，以免导致测量不准。

1.3 结构尺寸

PZ-VL53L0X 激光测距模块尺寸结构如下图所示：



2 硬件设计

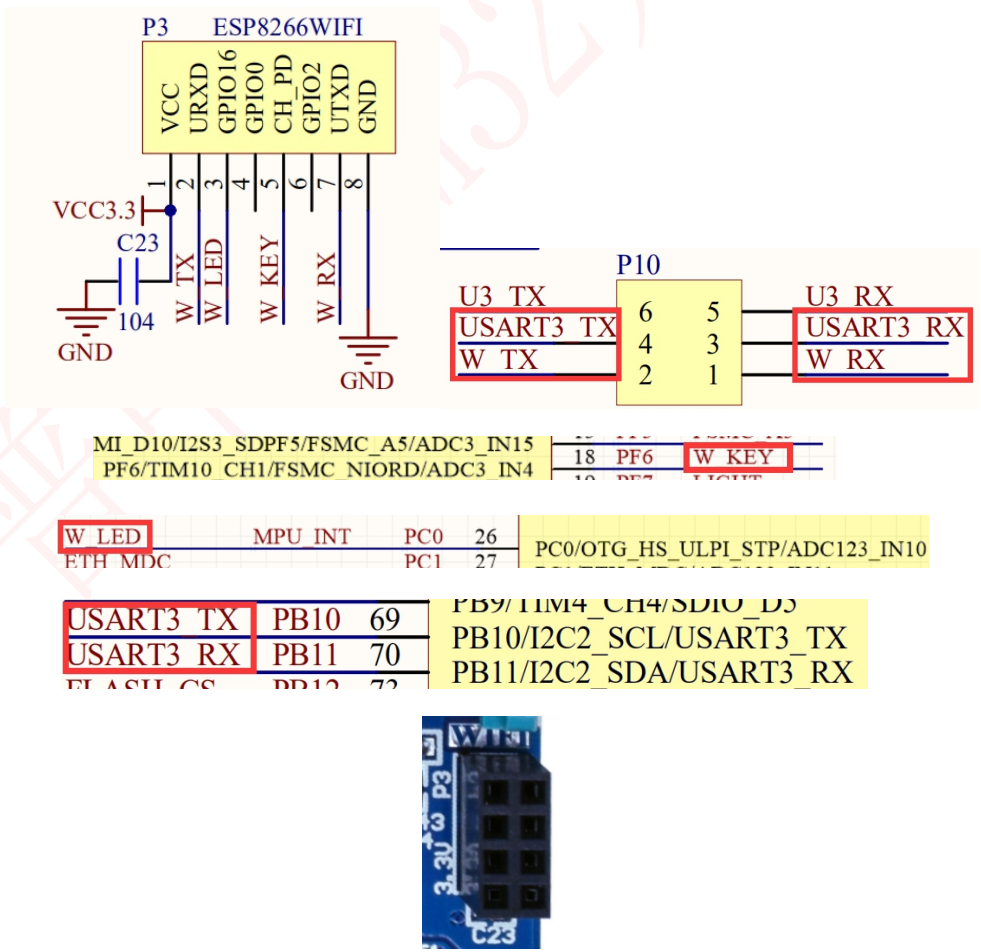
2.1 硬件准备

本实验所需要的硬件资源如下：

- ①F407 最小系统&天马&麒麟开发板 1 个
- ②TFTLCD 模块（不用彩屏也可用串口输出）
- ③PZ-VL53L0X 模块 1 个
- ④USB 线一条（用于供电和模块与电脑串口调试助手通信）

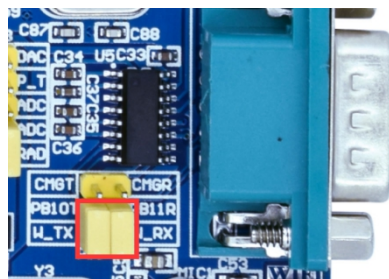
2.2 模块与开发板连接

PZ-VL53L0X 模块可直接与天马&麒麟开发板板载的 WIFI&蓝牙模块接口(P3)进行连接，该接口与 MCU 连接原理图（以麒麟为例）如下图所示：



从上图看出,WIFI 接口与 STM32 使用时,必须将 P10 的 USART3_TX(PB10

和 W_TX 以及 USART3_RX (PB11) 和 W_RX 连接, 才能完成和 STM32 的连接。默认天马&麒麟的 P10 端子上有黄色跳线帽, 直接短接到对应端即可。如下图所示:



对于 F407 最小系统, 是没有 WIFI 接口的, 所以需要通过杜邦线将模块管脚与上述 STM32 的 IO 口连接即可。

PZ-VL53L0X 激光测距模块与 STM32 的 IO 口连接关系如下: (PZ-VL53L0X → STM32F407)

VCC→3.3V
GND→GND
SCL→PB10
SDA→PB11
INT→PF6
XSH→PC0

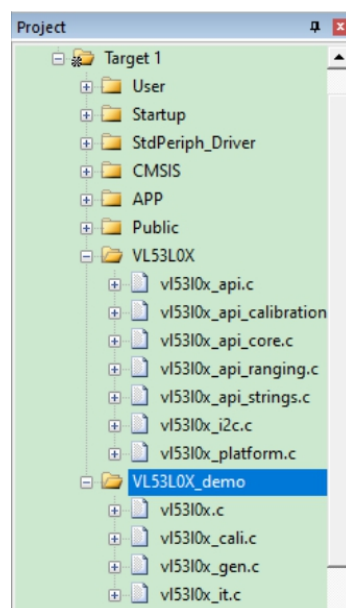
注意: 使用天马&麒麟开发板连接模块时, 模块插入 WIFI 接口处请注意方向, 可将模块电源与 WIFI 接口丝印电源对应即可。

3 软件设计

本实验在 F407 最小系统&天马&麒麟开发板的 FSMC-TFTLCD 显示实验基础上进行修改, 在 APP 文件夹内新建了 vl53l0x 文件夹, vl53l0x 文件夹内新建 core 文件夹, 存放官方移植的 VL53L0X API 驱动文件, vl53l0x_api.c、vl53l0x_api_calibration.c、vl53l0x_api_core.c 等文件。创建 platform 文件夹, 存放 VL53L0X 底层接口驱动文件, vl53l0x_i2c.c、vl53l0x_platform.c 等文件。最后创建 demo 文件夹, 创建 vl53l0x.c、vl53l0x_it.c、vl53l0x_cali.c、vl53l0x_gen.c 等文件

在工程目录添加 VL53L0X 和 VL53L0X_demo 分组, 将 core 和 platform 文件夹内的 C 文件添加到 VL53L0X 分组内, 接着将 demo 文件夹内的 C 文件

添加到 VL53L0X_demo 分组内，最后添加 core、platform 和 demo 文件夹到头文件包含路径。最终的工程如下图所示：



本例程由于代码量较多，我们仅对部分代码（vl53l0x_platform.c、vl53l0x.c、vl53l0x_it.c、vl53l0x_cal.c、vl53l0x_gen.c），以及 main 函数进行讲解。

3.1 vl53l0x_platform.c

vl53l0x_platform.c，该文件为底层驱动函数，它调用了 vl53l0x_i2c.c 文件的 iic 读写数据函数，实现了对 VL53L0X 读写操作，同时文件也实现对底层延时的功能，具体代码如下：

```
1.  #include "vl53l0x_platform.h"
2.
3.
4.
5.  //VL53L0X 连续写数据
6.  //Dev:设备 I2C 参数结构体
7.  //index:偏移地址
8.  //pdata:数据指针
9.  //count:长度
10. //返回值: 0:成功
11. //      其他:错误
12. VL53L0X_Error VL53L0X_WriteMulti(VL53L0X_DEV Dev, uint8_t index, uint8_t *pdata,uint32_t count)
13. {
```



```

14.
15.     VL53L0X_Error Status = VL53L0X_ERROR_NONE;
16.
17.     int32_t status_int = 0;
18.
19.     uint8_t deviceAddress;
20.
21.     if(count >=VL53L0X_MAX_I2C_XFER_SIZE)
22.     {
23.         Status = VL53L0X_ERROR_INVALID_PARAMS;
24.     }
25.
26.     deviceAddress = Dev->I2cDevAddr;
27.
28.     status_int = VL53L0X_write_multi(deviceAddress, index, pdata, count);
29.
30.     if(status_int !=0)
31.         Status = VL53L0X_ERROR_CONTROL_INTERFACE;
32.
33.     return Status;
34.
35. }
36.
37. //VL53L0X 连续读数据
38. //Dev:设备 I2C 参数结构体
39. //index:偏移地址
40. //pdata:数据指针
41. //count:长度
42. //返回值: 0:成功
43. //      其他:错误
44. VL53L0X_Error VL53L0X_ReadMulti(VL53L0X_DEV Dev, uint8_t index, uint8_t *pdata,uint32_t count)
45. {
46.
47.     VL53L0X_Error Status = VL53L0X_ERROR_NONE;
48.
49.     int32_t status_int;
50.
51.     uint8_t deviceAddress;
52.
53.     if(count >=VL53L0X_MAX_I2C_XFER_SIZE)
54.     {
55.         Status = VL53L0X_ERROR_INVALID_PARAMS;
56.     }

```

```

57.
58.     deviceAddress = Dev->I2cDevAddr;
59.
60.     status_int = VL53L0X_read_multi(deviceAddress, index, pdata, count);
61.
62.     if(status_int!=0)
63.         Status = VL53L0X_ERROR_CONTROL_INTERFACE;
64.
65.     return Status;
66.
67. }
68.
69. //VL53L0X 写单字节寄存器
70. //Dev:设备 I2C 参数结构体
71. //index:偏移地址
72. //pdata:数据指针
73. //count:长度
74. //返回值: 0:成功
75. //      其他:失败
76. VL53L0X_Error VL53L0X_WrByte(VL53L0X_DEV Dev, uint8_t index, uint8_t data)
77. {
78.
79.     VL53L0X_Error Status = VL53L0X_ERROR_NONE;
80.     int32_t status_int;
81.     uint8_t deviceAddress;
82.
83.     deviceAddress = Dev->I2cDevAddr;
84.
85.     status_int = VL53L0X_write_byte(deviceAddress,index,data);
86.
87.     if(status_int!=0)
88.         Status = VL53L0X_ERROR_CONTROL_INTERFACE;
89.
90.     return Status;
91.
92. }
93.
94. //VL53L0X 写字（2 字节）寄存器
95. //Dev:设备 I2C 参数结构体
96. //index:偏移地址
97. //pdata:数据指针
98. //count:长度
99. //返回值: 0:成功
100. //      其他:失败

```

```

101. VL53L0X_Error VL53L0X_WrWord(VL53L0X_DEV Dev, uint8_t index, uint16_t data)
102. {
103.
104.     VL53L0X_Error Status = VL53L0X_ERROR_NONE;
105.     int32_t status_int;
106.     uint8_t deviceAddress;
107.
108.     deviceAddress = Dev->I2cDevAddr;
109.
110.     status_int = VL53L0X_write_word(deviceAddress, index, data);
111.
112.     if(status_int!=0)
113.         Status = VL53L0X_ERROR_CONTROL_INTERFACE;
114.
115.     return Status;
116.
117. }
118.
119. //VL53L0X 写双字（4 字节）寄存器
120. //Dev:设备 I2C 参数结构体
121. //index:偏移地址
122. //pdata:数据指针
123. //count:长度
124. //返回值：0:成功
125. //      其他:失败
126. VL53L0X_Error VL53L0X_WrDWord(VL53L0X_DEV Dev, uint8_t index, uint32_t data)
127. {
128.
129.     VL53L0X_Error Status = VL53L0X_ERROR_NONE;
130.     int32_t status_int;
131.     uint8_t deviceAddress;
132.
133.     deviceAddress = Dev->I2cDevAddr;
134.
135.     status_int = VL53L0X_write_dword(deviceAddress, index, data);
136.
137.     if (status_int != 0)
138.         Status = VL53L0X_ERROR_CONTROL_INTERFACE;
139.
140.     return Status;
141.
142. }
143.
144. //VL53L0X 威胁安全更新(读/修改/写)单字节寄存器

```

```

145. //Dev:设备 I2C 参数结构体
146. //index:偏移地址
147. //AndData:8 位与数据
148. //OrData:8 位或数据
149. //返回值: 0:成功
150. //      其他:错误
151. VL53L0X_Error VL53L0X_UpdateByte(VL53L0X_DEV Dev, uint8_t index, uint8_t AndData, uint8_t OrData)
152. {
153.
154.     VL53L0X_Error Status = VL53L0X_ERROR_NONE;
155.     int32_t status_int;
156.     uint8_t deviceAddress;
157.     uint8_t data;
158.
159.     deviceAddress = Dev->I2cDevAddr;
160.
161.     status_int = VL53L0X_read_byte(deviceAddress,index,&data);
162.
163.     if(status_int!=0)
164.         Status = VL53L0X_ERROR_CONTROL_INTERFACE;
165.
166.     if(Status == VL53L0X_ERROR_NONE)
167.     {
168.         data = (data & AndData) | OrData;
169.         status_int = VL53L0X_write_byte(deviceAddress, index,data);
170.
171.
172.         if(status_int !=0)
173.             Status = VL53L0X_ERROR_CONTROL_INTERFACE;
174.
175.     }
176.
177.     return Status;
178.
179. }
180.
181. //VL53L0X 读单字节寄存器
182. //Dev:设备 I2C 参数结构体
183. //index:偏移地址
184. //pdata:数据指针
185. //count:长度
186. //返回值: 0:成功
187. //      其他:错误

```

```

188. VL53L0X_Error VL53L0X_RdByte(VL53L0X_DEV Dev, uint8_t index, uint8_t *data)
189. {
190.
191.     VL53L0X_Error Status = VL53L0X_ERROR_NONE;
192.     int32_t status_int;
193.     uint8_t deviceAddress;
194.
195.     deviceAddress = Dev->I2cDevAddr;
196.
197.     status_int = VL53L0X_read_byte(deviceAddress, index, data);
198.
199.     if(status_int !=0)
200.         Status = VL53L0X_ERROR_CONTROL_INTERFACE;
201.
202.     return Status;
203.
204. }
205.
206. //VL53L0X 读字（2 字节）寄存器
207. //Dev:设备 I2C 参数结构体
208. //index:偏移地址
209. //pdata:数据指针
210. //count:长度
211. //返回值：0:成功
212. //      其他:错误
213. VL53L0X_Error VL53L0X_RdWord(VL53L0X_DEV Dev, uint8_t index, uint16_t *data)
214. {
215.
216.     VL53L0X_Error Status = VL53L0X_ERROR_NONE;
217.     int32_t status_int;
218.     uint8_t deviceAddress;
219.
220.     deviceAddress = Dev->I2cDevAddr;
221.
222.     status_int = VL53L0X_read_word(deviceAddress, index, data);
223.
224.     if(status_int !=0)
225.         Status = VL53L0X_ERROR_CONTROL_INTERFACE;
226.
227.     return Status;
228.
229. }
230.
231. //VL53L0X 读双字（4 字节）寄存器

```

```

232. //Dev:设备 I2C 参数结构体
233. //index:偏移地址
234. //pdata:数据指针
235. //count:长度
236. //返回值: 0:成功
237. //      其他:错误
238. VL53L0X_Error VL53L0X_RdDWord(VL53L0X_DEV Dev, uint8_t index, uint32_t *data)
239. {
240.
241.     VL53L0X_Error Status = VL53L0X_ERROR_NONE;
242.     int32_t status_int;
243.     uint8_t deviceAddress;
244.
245.     deviceAddress = Dev->I2cDevAddr;
246.
247.     status_int = VL53L0X_read_dword(deviceAddress, index, data);
248.
249.     if (status_int != 0)
250.         Status = VL53L0X_ERROR_CONTROL_INTERFACE;
251.
252.     return Status;
253. }
254.
255. //VL53L0X 底层延时函数
256. //Dev:设备 I2C 参数结构体
257. //返回值: 0:成功
258. //      其他:错误
259. #define VL53L0X_POLLINGDELAY_LOOPNB 250
260. VL53L0X_Error VL53L0X_PollingDelay(VL53L0X_DEV Dev)
261. {
262.
263.     VL53L0X_Error status = VL53L0X_ERROR_NONE;
264.     volatile uint32_t i;
265.
266.     for(i=0;i<VL53L0X_POLLINGDELAY_LOOPNB;i++){
267.         //Do nothing
268.
269.     }
270.
271.     return status;
272. }

```

以上就是 vl53l0x_platform.c 文件的全部代码。都是 VL53L0X 底层操作的函数，比较简单，这里我们只挑 VL53L0X_WriteMult 和 VL53L0X_ReadMulti

两个函数说下，对于其他的函数这里我们就不做讲解了。VL53L0X_WriteMulti() 函数，该函数用于指定器件和地址，实现连续写数据，被 VL53L0X 的 API 函数所调用。而 VL53L0X_ReadMulti() 函数，该函数用于指定器件和地址，实现连续读数据，同样也是被 VL53L0X 的 API 函数所调用。

3.2 vl53l0x.c

vl53l0x.c 文件，该文件主要实现 vl53l0x 设备初始化、I2C 地址设置等函数。代码比较多，这里就不全部列出来了，仅介绍几个重要的函数。

首先是：print_pal_error 函数，该函数代码如下：

```
1. //API 错误信息打印
2. //Status: 详情看 VL53L0X_Error 参数的定义
3. void print_pal_error(VL53L0X_Error Status)
4. {
5.
6.     char buf[VL53L0X_MAX_STRING_LENGTH];
7.
8.     VL53L0X_GetPalErrorString(Status,buf);//根据 Status 状态获取错误信息字符串
9.
10.    printf("API Status: %i : %s\r\n",Status, buf);//打印状态和错误信息
11.
12. }
```

该函数调用了 VL53L0X_GetPalErrorString() API 函数，该函数可根据传入的 Status 状态值，获取状态值的信息字符串。根据字符串分析，可以快速定位错误问题点，方便调试。

然后我们看下 vl53l0x_Addr_set 函数，该函数代码如下：

```
1. //配置 VL53L0X 设备 I2C 地址
2. //dev: 设备 I2C 参数结构体
3. //newaddr: 设备新 I2C 地址
4. VL53L0X_Error vl53l0x_Addr_set(VL53L0X_Dev_t *dev,uint8_t newaddr)
5. {
6.     uint16_t Id;
7.     uint8_t FinalAddress;
8.     VL53L0X_Error Status = VL53L0X_ERROR_NONE;
9.     u8 sta=0x00;
10.
11.     FinalAddress = newaddr;
12. }
```



```

13.     if(FinalAddress==dev->I2cDevAddr)//新设备 I2C 地址与旧地址一致,直接退出
14.         return VL53L0X_ERROR_NONE;
15.     //在进行第一个寄存器访问之前设置 I2C 标准模式(400Khz)
16.     Status = VL53L0X_WrByte(dev,0x88,0x00);
17.     if(Status!=VL53L0X_ERROR_NONE)
18.     {
19.         sta=0x01;//设置 I2C 标准模式出错
20.         goto set_error;
21.     }
22.     //尝试使用默认的 0x52 地址读取一个寄存器
23.     Status = VL53L0X_RdWord(dev, VL53L0X_REG_IDENTIFICATION_MODEL_ID, &Id);
24.     if(Status!=VL53L0X_ERROR_NONE)
25.     {
26.         sta=0x02;//读取寄存器出错
27.         goto set_error;
28.     }
29.     if(Id == 0xEEAA)
30.     {
31.         //设置设备新的 I2C 地址
32.         Status = VL53L0X_SetDeviceAddress(dev,FinalAddress);
33.         if(Status!=VL53L0X_ERROR_NONE)
34.         {
35.             sta=0x03;//设置 I2C 地址出错
36.             goto set_error;
37.         }
38.         //修改参数结构体的 I2C 地址
39.         dev->I2cDevAddr = FinalAddress;
40.         //检查新的 I2C 地址读写是否正常
41.         Status = VL53L0X_RdWord(dev, VL53L0X_REG_IDENTIFICATION_MODEL_ID, &Id);
42.         if(Status!=VL53L0X_ERROR_NONE)
43.         {
44.             sta=0x04;//新 I2C 地址读写出错
45.             goto set_error;
46.         }
47.     }
48.     set_error:
49.     if(Status!=VL53L0X_ERROR_NONE)
50.     {
51.         print_pal_error(Status);//打印错误信息
52.     }
53.     if(sta!=0)
54.         printf("sta:0x%x\r\n",sta);
55.     return Status;
56. }

```

该函数实现对 vl53l0x 设备 I2C 地址的修改。设备初次上电默认 I2C 地址为 0x52，在修改前，我们先用旧地址读取寄存器的值，读取寄存器成功，保证通讯正常了，然后再调用 VL53L0X_SetDeviceAddress() API 函数设置新的地址，最后用新设置的 I2C 地址去读取寄存器的值，若读取寄存器成功，则表示 vl53l0x 新的 I2C 地址修改成功。

最后看下 vl53l0x_init 函数，该函数代码如下：

```
1. //初始化 vl53l0x
2. //dev:设备 I2C 参数结构体
3. VL53L0X_Error vl53l0x_init(VL53L0X_Dev_t *dev)
4. {
5.     GPIO_InitTypeDef GPIO_InitStructure;
6.     VL53L0X_Error Status = VL53L0X_ERROR_NONE;
7.     VL53L0X_Dev_t *pMyDevice = dev;
8.
9.     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC,ENABLE); //先使能外设 IO PORTC 时钟
10.
11.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0; //端口配置
12.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT; //普通输出模式
13.     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP; //推挽输出
14.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz; //100MHz
15.     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP; //上拉
16.     GPIO_Init(GPIOC, &GPIO_InitStructure); //初始化
17.
18.     pMyDevice->I2cDevAddr = VL53L0X_Addr; //I2C 地址(上电默认 0x52)
19.     pMyDevice->comms_type = 1; //I2C 通信模式
20.     pMyDevice->comms_speed_khz = 400; //I2C 通信速率
21.
22.     VL53L0X_i2c_init(); //初始化 IIC 总线
23.
24.     VL53L0X_Xshut=0; //失能 VL53L0X
25.     delay_ms(30);
26.     VL53L0X_Xshut=1; //使能 VL53L0X, 让传感器处于工作
27.     delay_ms(30);
28.
29.     vl53l0x_Addr_set(pMyDevice,0x54); //设置 VL53L0X 传感器 I2C 地址
30.     if(Status!=VL53L0X_ERROR_NONE) goto error;
31.     Status = VL53L0X_DataInit(pMyDevice); //设备初始化
32.     if(Status!=VL53L0X_ERROR_NONE) goto error;
33.     delay_ms(2);
34.     Status = VL53L0X_GetDeviceInfo(pMyDevice,&vl53l0x_dev_info); //获取设备 ID 信息
35.     if(Status!=VL53L0X_ERROR_NONE) goto error;
```

```

36.
37.     AT24CXX_Read(0, (u8*)&V15310x_data, sizeof(_v15310x_adjust)); //读取 24c02 保存的校准数据, 若已
    校准 V15310x_data.adjustok==0xAA
38.     if(V15310x_data.adjustok==0xAA) //已校准
39.         AdjustOK=1;
40.     else //没校准
41.         AdjustOK=0;
42.
43.     error:
44.     if(Status!=VL53L0X_ERROR_NONE)
45.     {
46.         print_pal_error(Status); //打印错误信息
47.         return Status;
48.     }
49.
50.     return Status;
51. }

```

该函数为 v15310x 设备初始化函数，函数入口参数为 dev，该参数用来保存 I2C 的设备信息，如地址和速度大小。在函数中，先对用到的引脚进行时钟等功能配置初始化，然后保存设备默认 I2C 地址（0X52）和其他的参数值到 pMyDevice 结构体中，接着初始化 VL53L0X 设备 I2C 引脚和使能 v15310x 设备，让设备处于工作状态，然后修改设备 I2C 地址，修改成功后，调用设备初始化 API 函数对 VL53L0X 设备进行初始化，等初始化结束后，调用信息获取 API 函数获取其设备 ID 信息，将 ID 信息保存到 v15310x_dev_info 信息结构体中。由于测试实验使用到 24C02 保存传感器校准的数据，所以在初始化最后会对 24C02 进行校准数据的读取，读取到标志为 0XAA，表示传感器上次已校准过，并且标志 AdjustOK 变量为 1。

3.3 vl53l0x_cali.c

vl53l0x_cali.c，该文件主要实现传感器的校准，该文件下代码比较多，这里就不全部列出来了，仅介绍传感器校准函数 vl53l0x_adjust()。

vl53l0x_adjust() 函数，代码如下：（注：由于代码较多，省略了部分非重要代码）

```

1.     #include "vl53l0x_cali.h"
2.
3.

```

```

4.
5.     _vl53l0x_adjust VL53l0x_adjust; //校准数据 24c02 写缓存区(用于在校准模式校准数据写入 24c02)
6.     _vl53l0x_adjust VL53l0x_data;   //校准数据 24c02 读缓存区(用于系统初始化时向 24C02 读取数据)
7.
8.     #define adjust_num 5//校准错误次数
9.
10.    //VL53L0X 校准函数
11.    //dev:设备 I2C 参数结构体
12.    VL53L0X_Error vl53l0x_adjust(VL53L0X_Dev_t *dev)
13.    {
14.
15.        VL53L0X_DeviceError Status = VL53L0X_ERROR_NONE;
16.        uint32_t refSpadCount = 7;
17.        uint8_t  isApertureSpads = 0;
18.        uint32_t XTalkCalDistance = 100;
19.        uint32_t XTalkCompensationRateMegaCps;
20.        uint32_t CalDistanceMilliMeter = 100<<16;
21.        int32_t  OffsetMicroMeter = 30000;
22.        uint8_t  VhvSettings = 23;
23.        uint8_t  PhaseCal = 1;
24.        u8 i=0;
25.
26.        VL53L0X_StaticInit(dev);//数值恢复默认,传感器处于空闲状态
27.        LED1=0;
28.        //SPADS 校准-----
29.        spads:
30.        delay_ms(10);
31.        printf("The SPADS Calibration Start...\r\n");
32.        Status = VL53L0X_PerformRefSpadManagement(dev,&refSpadCount,&isApertureSpads);//执行参考
        Spad 管理
33.        if(Status == VL53L0X_ERROR_NONE)
34.        {
35.            printf("refSpadCount = %d\r\n",refSpadCount);
36.            VL53l0x_adjust.refSpadCount = refSpadCount;
37.            printf("isApertureSpads = %d\r\n",isApertureSpads);
38.            VL53l0x_adjust.isApertureSpads = isApertureSpads;
39.            printf("The SPADS Calibration Finish...\r\n\r\n");
40.            i=0;
41.        }
42.        else
43.        {
44.            i++;
45.            if(i==adjust_num) return Status;
46.            printf("SPADS Calibration Error,Restart this step\r\n");

```

```

47.         goto spads;
48.     }
49.     //设备参考校准-----
50.     ref:
51.     delay_ms(10);
52.     printf("The Ref Calibration Start...\r\n");
53.     Status = VL53L0X_PerformRefCalibration(dev,&VhvSettings,&PhaseCal);//Ref 参考校准
54.     if(Status == VL53L0X_ERROR_NONE)
55.     {
56.         printf("VhvSettings = %d\r\n",VhvSettings);
57.         VL53L0X_adjust.VhvSettings = VhvSettings;
58.         printf("PhaseCal = %d\r\n",PhaseCal);
59.         VL53L0X_adjust.PhaseCal = PhaseCal;
60.         printf("The Ref Calibration Finish...\r\n\r\n");
61.         i=0;
62.     }
63.     else
64.     {
65.         i++;
66.         if(i==adjust_num) return Status;
67.         printf("Ref Calibration Error,Restart this step\r\n");
68.         goto ref;
69.     }
70.     //偏移校准-----
71.     offset:
72.     delay_ms(10);
73.     printf("Offset Calibration:need a white target,in black space,and the distance keep 100
mm!\r\n");
74.     printf("The Offset Calibration Start...\r\n");
75.     Status = VL53L0X_PerformOffsetCalibration(dev,CalDistanceMilliMeter,&OffsetMicroMeter);
//偏移校准
76.     if(Status == VL53L0X_ERROR_NONE)
77.     {
78.         printf("CalDistanceMilliMeter = %d mm\r\n",CalDistanceMilliMeter);
79.         VL53L0X_adjust.CalDistanceMilliMeter = CalDistanceMilliMeter;
80.         printf("OffsetMicroMeter = %d mm\r\n",OffsetMicroMeter);
81.         VL53L0X_adjust.OffsetMicroMeter = OffsetMicroMeter;
82.         printf("The Offset Calibration Finish...\r\n\r\n");
83.         i=0;
84.     }
85.     else
86.     {
87.         i++;
88.         if(i==adjust_num) return Status;

```

```

89.         printf("Offset Calibration Error,Restart this step\r\n");
90.         goto offset;
91.     }
92.     //串扰校准-----
93.     xtalk:
94.     delay_ms(20);
95.     printf("Cross Talk Calibration:need a grey target\r\n");
96.     printf("The Cross Talk Calibration Start...\r\n");
97.     Status = VL53L0X_PerformXTalkCalibration(dev,XTalkCalDistance,&XTalkCompensationRateMegaCps);//串扰校准
98.     if(Status == VL53L0X_ERROR_NONE)
99.     {
100.         printf("XTalkCalDistance = %d mm\r\n",XTalkCalDistance);
101.         VL53L0X_adjust.XTalkCalDistance = XTalkCalDistance;
102.         printf("XTalkCompensationRateMegaCps = %d\r\n",XTalkCompensationRateMegaCps);
103.         VL53L0X_adjust.XTalkCompensationRateMegaCps = XTalkCompensationRateMegaCps;
104.         printf("The Cross Talk Calibration Finish...\r\n\r\n");
105.         i=0;
106.     }
107.     else
108.     {
109.         i++;
110.         if(i==adjust_num) return Status;
111.         printf("Cross Talk Calibration Error,Restart this step\r\n");
112.         goto xtalk;
113.     }
114.     LED1=1;
115.     printf("All the Calibration has Finished!\r\n");
116.     printf("Calibration is successful!!\r\n");
117.
118.     VL53L0X_adjust.adjustok = 0xAA;//校准成功
119.     AT24CXX_Write(0,(u8*)&VL53L0X_adjust,sizeof(_vl53l0x_adjust));//将校准数据保存到 24c02
120.     memcpy(&VL53L0X_data,&VL53L0X_adjust,sizeof(_vl53l0x_adjust));//将校准数据复制到 VL53L0X_data 结构体
121.     return Status;
122. }

```

该函数为 vl53l0x 传感器校准函数，实现对传感器的 SPAD（接收端）、Ref 参考设备、距离偏移、以及串扰进行校准。这里我们定义了 VL53L0X_adjust 校准数据写缓冲区和 VL53L0X_data 校准数据读缓冲区的全局变量结构体，将每项校准后的数据保存 VL53L0X_adjust 结构体变量中，同时标志 VL53L0X_adjust.adjustok 标志位为 0xAA，以表示校准成功。校准的数据会保

存在 24C02 上，同时也将当前的校准数据同步更新到 VL5310x_data 校准数据读缓冲区中。

3.4 vl53l0x_gen.c

vl53l0x_gen.c 文件，该文件主要实现单次测量、精度模式配置等函数，文件代码比较多，这里就不全部列出来了，仅介绍几个重要的函数。

vl53l0x_set_mode() 函数，该函数代码如下：

```
1. //VL53L0X 测量模式配置
2. //dev:设备 I2C 参数结构体
3. //mode: 0:默认;1:高精度;2:长距离;3:高速
4. VL53L0X_Error vl53l0x_set_mode(VL53L0X_Dev_t *dev,u8 mode)
5. {
6.
7.     VL53L0X_Error status = VL53L0X_ERROR_NONE;
8.     uint8_t VhvSettings;
9.     uint8_t PhaseCal;
10.    uint32_t refSpadCount;
11.    uint8_t isApertureSpads;
12.
13.    vl53l0x_reset(dev);//复位 vl53l0x(频繁切换工作模式容易导致采集距离数据不准，需加上这一代码)
14.    status = VL53L0X_StaticInit(dev);
15.
16.    if(AjustOK!=0)//已校准好了,写入校准值
17.    {
18.        status= VL53L0X_SetReferenceSpads(dev,VL53l0x_data.refSpadCount,VL53l0x_data.isApertureSpads);//设定 Spads 校准值
19.        if(status!=VL53L0X_ERROR_NONE) goto error;
20.        delay_ms(2);
21.        status= VL53L0X_SetRefCalibration(dev,VL53l0x_data.VhvSettings,VL53l0x_data.PhaseCal1);//设定 Ref 校准值
22.        if(status!=VL53L0X_ERROR_NONE) goto error;
23.        delay_ms(2);
24.        status=VL53L0X_SetOffsetCalibrationDataMicroMeter(dev,VL53l0x_data.OffsetMicroMeter);//设定偏移校准值
25.        if(status!=VL53L0X_ERROR_NONE) goto error;
26.        delay_ms(2);
27.        status=VL53L0X_SetXTalkCompensationRateMegaCps(dev,VL53l0x_data.XTalkCompensationRateMegaCps);//设定串扰校准值
28.        if(status!=VL53L0X_ERROR_NONE) goto error;
29.        delay_ms(2);
```

```

30.
31.     }
32.     else
33.     {
34.         status = VL53L0X_PerformRefCalibration(dev, &VhvSettings, &PhaseCal); //Ref 参考校准
35.         if(status!=VL53L0X_ERROR_NONE) goto error;
36.         delay_ms(2);
37.         status = VL53L0X_PerformRefSpadManagement(dev, &refSpadCount, &isApertureSpads); //执
            行参考 SPAD 管理
38.         if(status!=VL53L0X_ERROR_NONE) goto error;
39.         delay_ms(2);
40.     }
41.     status = VL53L0X_SetDeviceMode(dev, VL53L0X_DEVICEMODE_SINGLE_RANGING); // 使能单次测量模
            式
42.     if(status!=VL53L0X_ERROR_NONE) goto error;
43.     delay_ms(2);
44.     status = VL53L0X_SetLimitCheckEnable(dev, VL53L0X_CHECKENABLE_SIGMA_FINAL_RANGE, 1); // 使
            能 SIGMA 范围检查
45.     if(status!=VL53L0X_ERROR_NONE) goto error;
46.     delay_ms(2);
47.     status = VL53L0X_SetLimitCheckEnable(dev, VL53L0X_CHECKENABLE_SIGNAL_RATE_FINAL_RANGE, 1)
            ; //使能信号速率范围检查
48.     if(status!=VL53L0X_ERROR_NONE) goto error;
49.     delay_ms(2);
50.     status = VL53L0X_SetLimitCheckValue(dev, VL53L0X_CHECKENABLE_SIGMA_FINAL_RANGE, Mode_dat
            a[mode].sigmaLimit); //设定 SIGMA 范围
51.     if(status!=VL53L0X_ERROR_NONE) goto error;
52.     delay_ms(2);
53.     status = VL53L0X_SetLimitCheckValue(dev, VL53L0X_CHECKENABLE_SIGNAL_RATE_FINAL_RANGE, Mo
            de_data[mode].signalLimit); //设定信号速率范围范围
54.     if(status!=VL53L0X_ERROR_NONE) goto error;
55.     delay_ms(2);
56.     status = VL53L0X_SetMeasurementTimingBudgetMicroSeconds(dev, Mode_data[mode].timingBudg
            et); //设定完整测距最长时间
57.     if(status!=VL53L0X_ERROR_NONE) goto error;
58.     delay_ms(2);
59.     status = VL53L0X_SetVcselPulsePeriod(dev, VL53L0X_VCSEL_PERIOD_PRE_RANGE, Mode_data[mo
            de].preRangeVcselPeriod); //设定 VCSEL 脉冲周期
60.     if(status!=VL53L0X_ERROR_NONE) goto error;
61.     delay_ms(2);
62.     status = VL53L0X_SetVcselPulsePeriod(dev, VL53L0X_VCSEL_PERIOD_FINAL_RANGE, Mode_data[
            mode].finalRangeVcselPeriod); //设定 VCSEL 脉冲周期范围
63.
64.     error; //错误信息

```



```

65.     if(status!=VL53L0X_ERROR_NONE)
66.     {
67.         print_pal_error(status);
68.         LCD_Fill(30,140+20,300,300,WHITE);
69.         return status;
70.     }
71.     return status;
72.
73. }

```

vl53l0x_set_mode() 函数为单次测量精度模式配置函数，该函数在测量前，对 VL53L0X 传感器进行误差的校正，在使能单次测量模式后，配置工作精度模式。精度模式的配置根据函数参数 mode 变量决定，mode 值对应，0：默认，1：高精度，2：长距离，3：高速，根据全局数组 Mode_data[] 变量的写入，从而实现不同精度模式的配置，该全局数组 Mode_data[] 定义位置在 vl53l0x.c 文件下。在函数的 VL53L0X 运行前，我们做了复位，由于频繁精度模式的切换，容易导致采集的距离数据不准，所以才加上复位。

接着我们看下 vl53l0x_start_single_test() 函数，该函数代码如下：

```

1.  //VL53L0X 单次距离测量函数
2.  //dev:设备 I2C 参数结构体
3.  //pdata:保存测量数据结构体
4.  VL53L0X_Error vl53l0x_start_single_test(VL53L0X_Dev_t *dev,VL53L0X_RangingMeasurementData_t
    *pdata,char *buf)
5.  {
6.      VL53L0X_Error status = VL53L0X_ERROR_NONE;
7.      uint8_t RangeStatus;
8.
9.      status = VL53L0X_PerformSingleRangingMeasurement(dev, pdata);//执行单次测距并获取测距测量
    数据
10.     if(status !=VL53L0X_ERROR_NONE) return status;
11.
12.     RangeStatus = pdata->RangeStatus;//获取当前测量状态
13.     memset(buf,0x00,VL53L0X_MAX_STRING_LENGTH);
14.     VL53L0X_GetRangeStatusString(RangeStatus,buf);//根据测量状态读取状态字符串
15.
16.     Distance_data = pdata->RangeMilliMeter;//保存最近一次测距测量数据
17.
18.     return status;
19. }

```

该函数为单次测量函数，通过调用

VL53L0X_PerformSingleRangingMeasurement () API 函数启动单次测量, 该 API 函数为阻塞函数, 测量数据返回到 pdata 测量数据结构体上, 最后将获取测量结构体的测量数据赋值给 Distance_data 全局变量。

3.5 vl50l0x_it.c

vl50l0x_it.c 文件, 该文件主要实现连续测量 (中断模式)、精度配置等函数。文件代码比较多, 这里就不全部列出来了, 仅介绍连续测量函数:

vl53l0x_interrupt_start():

vl53l0x_interrupt_start() 函数, 该函数代码如下:

```
1. //vl53l0x 中断测量模式测试
2. //dev:设备 I2C 参数结构体
3. //mode: 0:默认;1:高精度;2:长距离;3:高速
4. void vl53l0x_interrupt_start(VL53L0X_Dev_t *dev,uint8_t mode)
5. {
6.     uint8_t VhvSettings;
7.     uint8_t PhaseCal;
8.     uint32_t refSpadCount;
9.     uint8_t isApertureSpads;
10.    VL53L0X_RangingMeasurementData_t RangingMeasurementData;
11.    static char buf[VL53L0X_MAX_STRING_LENGTH]; //测试模式字符串字符缓冲区
12.    VL53L0X_Error status=VL53L0X_ERROR_NONE; //工作状态
13.    u8 key;
14.
15.    exti_init(); //中断初始化
16.    LED1=1;
17.    mode_string(mode,buf); //显示当前配置的模式
18.    LCD_Fill(30,170,300,300,WHITE);
19.    FRONT_COLOR=RED; //设置字体为红色
20.    LCD_ShowString(30,140+30,300,16,16,"Interrupt Mode ");
21.    FRONT_COLOR=BLUE; //设置字体为蓝色
22.    LCD_ShowString(30,140+50,200,16,16,"KEY_UP: Exit the test ");
23.    LCD_ShowString(30,140+70,200,16,16,"Mode: ");
24.    LCD_ShowString(80,140+70,200,16,16,(u8*)buf);
25.    sprintf((char*)buf,"Thresh Low: %d mm ",Thresh_Low);
26.    LCD_ShowString(30,140+90,300,16,16,(u8*)buf);
27.    sprintf((char*)buf,"Thresh High: %d mm",Thresh_High);
28.    LCD_ShowString(30,140+110,300,16,16,(u8*)buf);
29.    LCD_ShowString(30,140+130,300,16,16,"Now value: mm");
30.
```

```

31.     vl53l0x_reset(dev); //复位 vl53l0x(频繁切换工作模式容易导致采集距离数据不准,需加上这一代码)
32.     status = VL53L0X_StaticInit(dev);
33.     if(status!=VL53L0X_ERROR_NONE) goto error;
34.
35.     if(AjustOK!=0) //已校准好了,写入校准值
36.     {
37.         status= VL53L0X_SetReferenceSpads(dev,Vl53l0x_data.refSpadCount,Vl53l0x_data.isApertureSpads); //设定 Spads 校准值
38.         if(status!=VL53L0X_ERROR_NONE) goto error;
39.         delay_ms(2);
40.         status= VL53L0X_SetRefCalibration(dev,Vl53l0x_data.VhvSettings,Vl53l0x_data.PhaseCal1); //设定 Ref 校准值
41.         if(status!=VL53L0X_ERROR_NONE) goto error;
42.         delay_ms(2);
43.         status=VL53L0X_SetOffsetCalibrationDataMicroMeter(dev,Vl53l0x_data.OffsetMicroMeter); //设定偏移校准值
44.         if(status!=VL53L0X_ERROR_NONE) goto error;
45.         delay_ms(2);
46.         status=VL53L0X_SetXTalkCompensationRateMegaCps(dev,Vl53l0x_data.XTalkCompensationRateMegaCps); //设定串扰校准值
47.         if(status!=VL53L0X_ERROR_NONE) goto error;
48.     }else
49.     {
50.         status = VL53L0X_PerformRefCalibration(dev, &VhvSettings, &PhaseCal); //Ref 参考校准
51.         if(status!=VL53L0X_ERROR_NONE) goto error;
52.         delay_ms(2);
53.         status = VL53L0X_PerformRefSpadManagement(dev, &refSpadCount, &isApertureSpads); //执行参考 SPAD 管理
54.         if(status!=VL53L0X_ERROR_NONE) goto error;
55.         delay_ms(2);
56.     }
57.     status = VL53L0X_SetDeviceMode(dev,VL53L0X_DEVICEMODE_CONTINUOUS_RANGING); //使能连续测量模式
58.     if(status!=VL53L0X_ERROR_NONE) goto error;
59.     delay_ms(2);
60.     status = VL53L0X_SetInterMeasurementPeriodMilliseconds(dev,Mode_data[mode].timingBudget); //设置内部周期测量时间
61.     if(status!=VL53L0X_ERROR_NONE) goto error;
62.     delay_ms(2);
63.     status = VL53L0X_SetLimitCheckEnable(dev,VL53L0X_CHECKENABLE_SIGMA_FINAL_RANGE,1); //使能 SIGMA 范围检查
64.     if(status!=VL53L0X_ERROR_NONE) goto error;
65.     delay_ms(2);

```

```

66.         status = VL53L0X_SetLimitCheckEnable(dev,VL53L0X_CHECKENABLE_SIGNAL_RATE_FINAL_RANGE,1)
           ;//使能信号速率范围检查
67.         if(status!=VL53L0X_ERROR_NONE) goto error;
68.         delay_ms(2);
69.         status = VL53L0X_SetLimitCheckValue(dev,VL53L0X_CHECKENABLE_SIGMA_FINAL_RANGE,Mode_data[mode].sigmaLimit);//设定 SIGMA 范围
70.         if(status!=VL53L0X_ERROR_NONE) goto error;
71.         delay_ms(2);
72.         status = VL53L0X_SetLimitCheckValue(dev,VL53L0X_CHECKENABLE_SIGNAL_RATE_FINAL_RANGE,Mode_data[mode].signalLimit);//设定信号速率范围范围
73.         if(status!=VL53L0X_ERROR_NONE) goto error;
74.         delay_ms(2);
75.         status = VL53L0X_SetMeasurementTimingBudgetMicroSeconds(dev,Mode_data[mode].timingBudget);//设定完整测距最长时间
76.         if(status!=VL53L0X_ERROR_NONE) goto error;
77.         delay_ms(2);
78.         status = VL53L0X_SetVcseLPulsePeriod(dev, VL53L0X_VCSEL_PERIOD_PRE_RANGE, Mode_data[mode].preRangeVcseLPulsePeriod);//设定 VCSEL 脉冲周期
79.         if(status!=VL53L0X_ERROR_NONE) goto error;
80.         delay_ms(2);
81.         status = VL53L0X_SetVcseLPulsePeriod(dev, VL53L0X_VCSEL_PERIOD_FINAL_RANGE, Mode_data[mode].finalRangeVcseLPulsePeriod);//设定 VCSEL 脉冲周期范围
82.         if(status!=VL53L0X_ERROR_NONE) goto error;
83.         delay_ms(2);
84.         status = VL53L0X_StopMeasurement(dev);//停止测量
85.         if(status!=VL53L0X_ERROR_NONE) goto error;
86.         delay_ms(2);
87.         status = VL53L0X_SetInterruptThresholds(dev,VL53L0X_DEVICEMODE_CONTINUOUS_RANGING,AlarmModes.ThreshLow, AlarmModes.ThreshHigh);//设定触发中断上、下限值
88.         if(status!=VL53L0X_ERROR_NONE) goto error;
89.         delay_ms(2);
90.         status = VL53L0X_SetGpioConfig(dev,0,VL53L0X_DEVICEMODE_CONTINUOUS_RANGING,AlarmModes.VL53L0X_Mode,VL53L0X_INTERRUPTPOLARITY_LOW);//设定触发中断模式 下降沿
91.         if(status!=VL53L0X_ERROR_NONE) goto error;
92.         delay_ms(2);
93.         status = VL53L0X_ClearInterruptMask(dev,0);//清除 VL53L0X 中断标志位
94.
95.         error;//错误信息
96.         if(status!=VL53L0X_ERROR_NONE)
97.         {
98.             print_pal_error(status);
99.             return ;
100.        }
101.

```

```

102.     alarm_flag = 0;
103.     VL53L0X_StartMeasurement(dev); //启动测量
104.     while(1)
105.     {
106.         key = KEY_Scan(0);
107.         if(key==KEY_UP_PRESS)
108.         {
109.             VL53L0X_ClearInterruptMask(dev,0); //清除 VL53L0X 中断标志位
110.             status = VL53L0X_StopMeasurement(dev); //停止测量
111.             LED2=1;
112.             break; //返回上一菜单
113.         }
114.         if(alarm_flag==1) //触发中断
115.         {
116.             alarm_flag=0;
117.             VL53L0X_GetRangingMeasurementData(dev,&RangingMeasurementData); //获取测量距离,并
            且显示距离
118.             printf("d: %3d mm\r\n",RangingMeasurementData.RangeMilliMeter);
119.             LCD_ShowxNum(110,140+130,RangingMeasurementData.RangeMilliMeter,4,16,0);
120.             delay_ms(70);
121.             VL53L0X_ClearInterruptMask(dev,0); //清除 VL53L0X 中断标志位
122.
123.         }
124.         delay_ms(30);
125.         LED2=!LED2;
126.
127.     }
128.
129. }

```

vl53l0x_interrupt_start() 函数为连续测量（中断模式）测试函数，该函数实现了连续测量（中断模式）距离。运行过程，首先对 VL53L0X 误差进行校正，然后配置工作的精度模式，由于使用了中断模式，需设置 GPIO1 中断引脚的触发模式和上下限距离值，这里我们是使用了 CROSSED_OUT 模式（即“测量距离<下限距离”或“测量距离>上限距离”时触发中断，获取测量数据。），中断配置好后，启动距离测量。当测量距离在窗口模式范围时，GPIO1 会产生中断，在中断服务函数（中断服务函数代码这里没有截取出来）将 alarm_flag 标志位置 1，最后根据标志位去读取测量的数据。

3.6 main.c

关于 VL53L0X 的例程代码，我们就介绍到这来，我们再来看看 main.c，该文件里面就一个 main 函数，main 函数代码如下：

```
1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "led.h"
4.  #include "usart.h"
5.  #include "tftlcd.h"
6.  #include "key.h"
7.  #include "24cxx.h"
8.  #include "vl53l0x.h"
9.
10.
11.  /*****
12.   * 函数名      : main
13.   * 函数功能    : 主函数
14.   * 输 入      : 无
15.   * 输 出      : 无
16.   *****/
17.  int main()
18.  {
19.      u8 i=0;
20.      u16 color=0;
21.
22.      SysTick_Init(168);
23.      NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
24.      LED_Init();
25.      USART1_Init(115200);
26.      TFTLCD_Init(); //LCD 初始化
27.      KEY_Init(); //按键初始化
28.      AT24CXX_Init(); //IIC 初始化
29.
30.      FRONT_COLOR=RED;
31.      LCD_ShowString(30,10,tftlcd_data.width,tftlcd_data.height,16,"PRECHIN-STM32F4");
32.      LCD_ShowString(30,30,tftlcd_data.width,tftlcd_data.height,16,"Sensor VL53L0X TEST");
33.      LCD_ShowString(30,50,tftlcd_data.width,tftlcd_data.height,16,"www.prechin.net");
34.      FRONT_COLOR=BLUE;
35.      while(AT24CXX_Check())//检测不到 24c02
36.      {
37.          LCD_ShowString(30,150,200,16,16,"24C02 Check Failed!");
38.          delay_ms(500);
```

```

39.      LCD_ShowString(30,150,200,16,16,"Please Check!   ");
40.      delay_ms(500);
41.      LED1=!LED1;//DS0 闪烁
42.  }
43.  while(1)
44.  {
45.      v15310x_test();//v15310x 测试
46.  }
47.  }

```

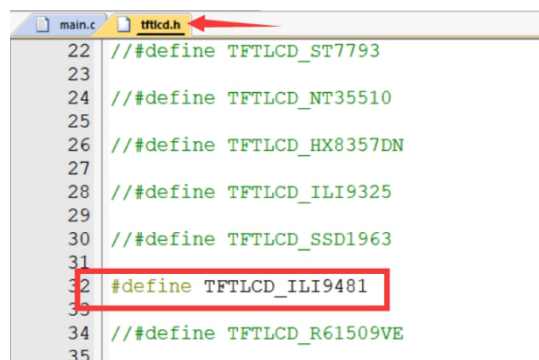
此部分代码比较简单，对用到的外设进行初始化，然后通过调用 `v15310x_test()`，进入 PZ-VL53L0X 模块的主测试程序，对 PZ-VL53L0X 的各项功能（校准测试，普通测量测试、中断测量测试）进行测试。

4 实验现象

首先，请先确保硬件都已经连接好：

- ①PZ-VL53L0X 模块与 F407 最小系统&天马&麒麟开发板连接（可参考硬件设计小节）
- ②开发板插上 TFTLCD 液晶，对于没有 TFTLCD 的用户，可通过串口输出。
- ③使用 USB 线给开发板供电。

注意：如果使用普中 TFTLCD 触摸屏，用户需根据手上彩屏右上角或背面驱动型号来设置程序中使能的 TFTLCD 驱动。在 `tftlcd.h` 头文件中开始处进行设置，比如我使用的 TFTLCD 是 ILI9481，那么就要把这个宏定义放开，其他的注释掉。如下所示：

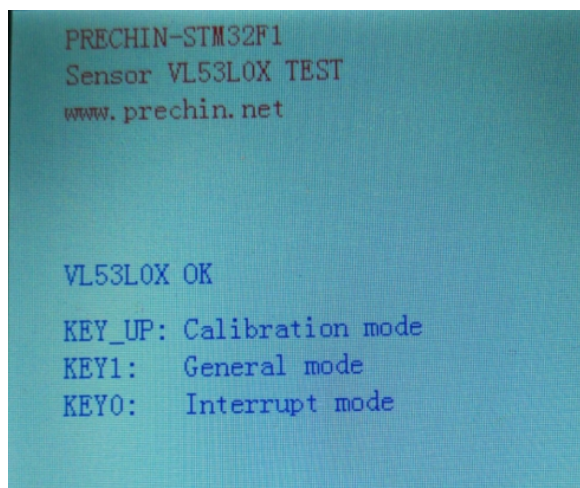


```

22  // #define TFTLCD_ST7793
23
24  // #define TFTLCD_NT35510
25
26  // #define TFTLCD_HX8357DN
27
28  // #define TFTLCD_ILI9325
29
30  // #define TFTLCD_SSD1963
31
32  #define TFTLCD_ILI9481
33
34  // #define TFTLCD_R61509VE
35

```

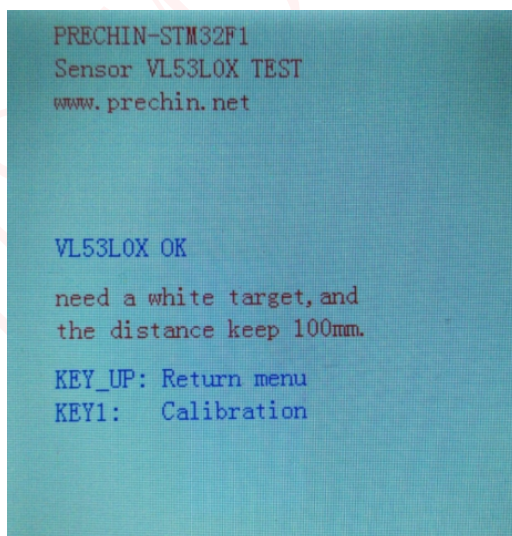
代码编译成功之后，我们将代码下载到 STM32 开发板上。（注意：以下的功能测试图片以天马开发板为展示）LCD 界面显示如下图所示：



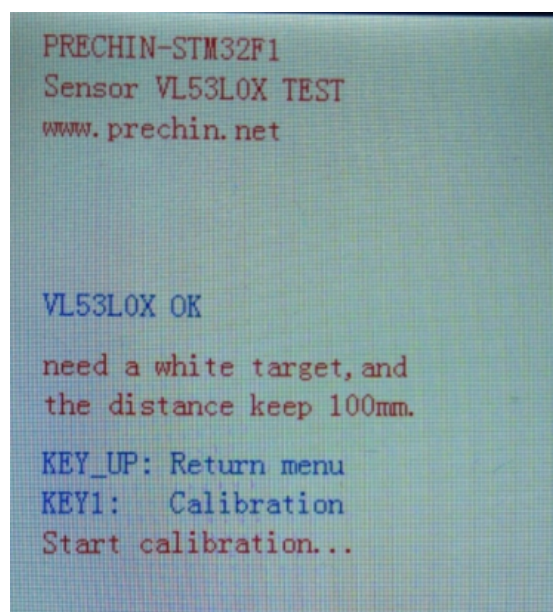
可以看到，LCD 屏幕显示 VL53L0X OK，表示 VL53L0X 传感器初始化通过，同时显示了 KEY_UP/KEY1/KEY0 测试功能选项，KEY_UP 校准测试、KEY1 普通测量测试、以及 KEY0 中断测量测试。

4.1 校准测试

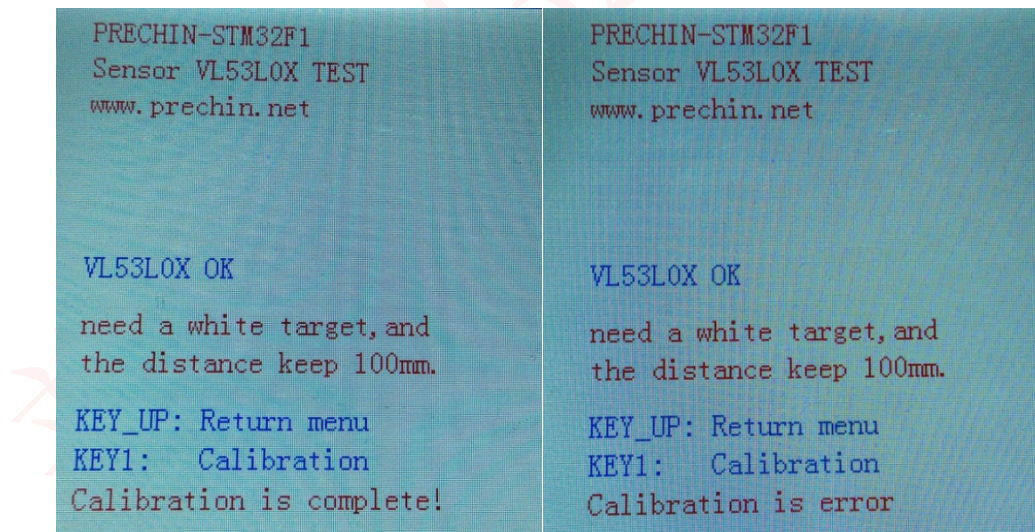
在主菜单界面，按 KEY_UP，则可进入此项测试，此项测试为校准 VL53L0X 传感器，校准测试主界面如下图所示：



LCD 屏幕显示校准需要一个白色目标物体，而且 VL53L0X 传感器与目标距离需保持 10 厘米（100mm）。当确认目标和距离无误后，按下 KEY1 执行校准。若不执行校准，则按下 KEY_UP 返回主菜单页面。按下 KEY1 按键后，这时会提示 Start Calibration，表示校准开始，



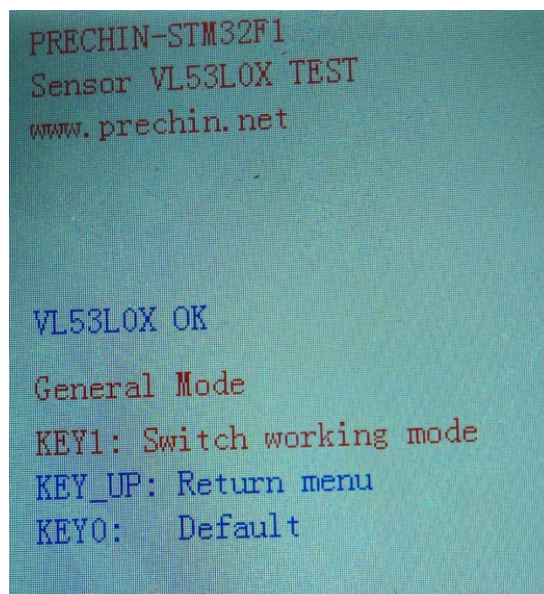
校准结束后，LCD 屏幕会显示校准状态，若显示 Calibration is complete!，则表示校准结束（成功），显示 Calibration is error，则表示校准失败，同时开发板 DS1 灯保持常亮以提示校准失败。（注意：校准失败有可能是当前 VL53L0X 传感器接收不到返回的激光信号，所以请检查模块与目标距离是否为 10 厘米）。校准状态的显示，如下图所示：



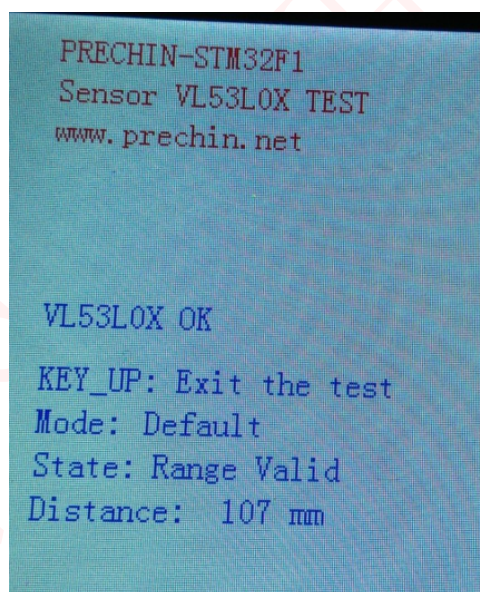
校准状态显示后，LCD 屏幕显示会自动返回主菜单页面。

4.2 普通测量测试

在主菜单页面，按 KEY1，可进入此项测试，此项可测试在单次测量工作模式下，实现在不同精度模式下进行距离的测量。普通测量测试主界面如下图所示：



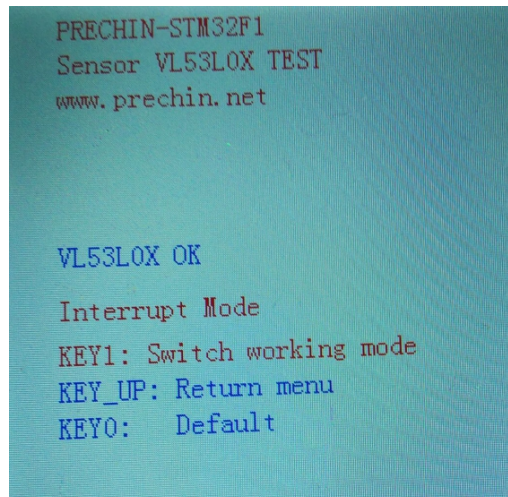
按 KEY1 可切换精度模式, 从上图可以看到, 当前设置为 Default 默认精度, 按 KEY0 进入测试, 测试的界面如下图所示。



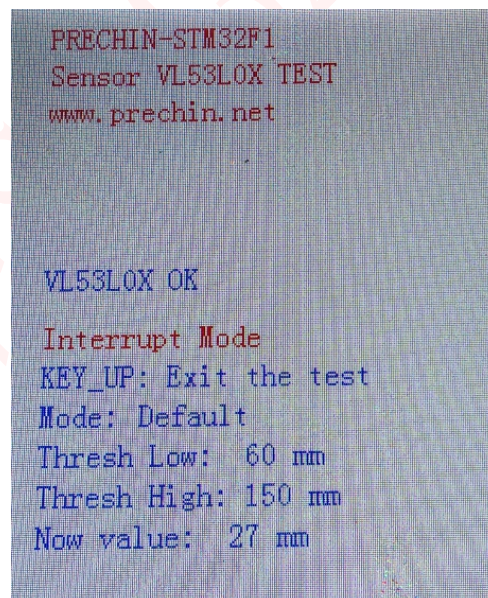
进入测试后, LCD 屏幕会显示当前工作的精度模式, 测量状态, 以及测量的距离。当测量的距离在有效的测量范围内时, State 状态会显示 Range Valid (范围有效), 超出范围则显示 Phase Fail (相位失效)。从图中看到, 当前测量的距离为 10.7 厘米 (107mm), 测量数据在有效范围内。单击 KEY_UP 会返回精度选择页面, 双击 KEY_UP 则会返回主菜单页面。

4.3 中断测量测试

在主菜单页面，按 KEY0，可进入此项测试，此项可测试在连续测量工作模式下，利用中断触发，实现在不同精度模式下进行距离的测量。中断测量测试主界面如下图所示：



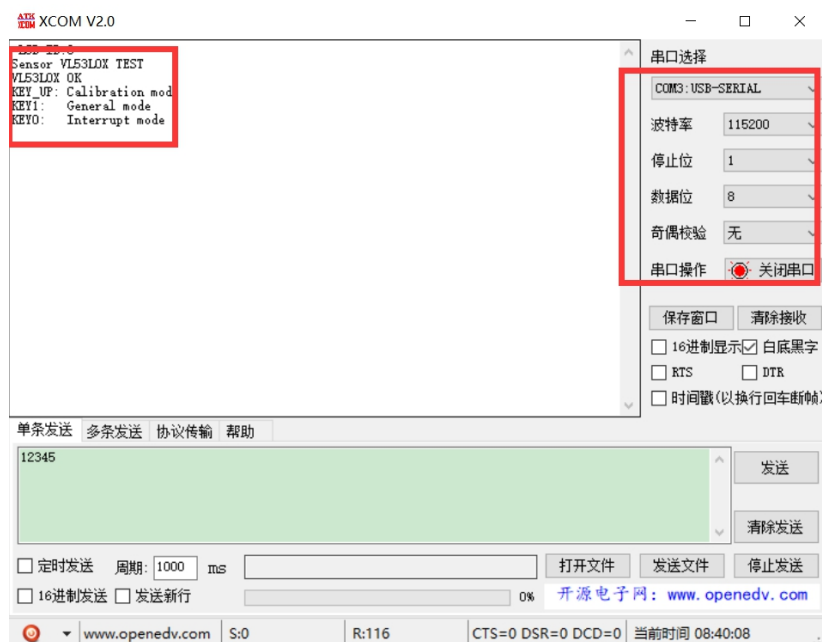
按 KEY1 可切换精度模式，从图中可以看到，当前设置的是 Default 默认精度，按 KEY0 可进入测试，测试的界面如下图所示。



进入测试后，LCD 屏幕会显示当前工作的精度模式，触发中断的上下限距离值，以及测量的距离。当测量的距离在上下限距离范围内时，测量中断不触发，距离数据不做更新变化。当距离小于下限距离值或大于上限距离值时，测量中断触发，测量的距离数据发生变化。从图中看到，当前测量的距离为 2.7 厘米（27mm）。单击 KEY_UP 会返回精度选择页面，双击 KEY_UP 则返回主菜单页面。

4.4 串口输出

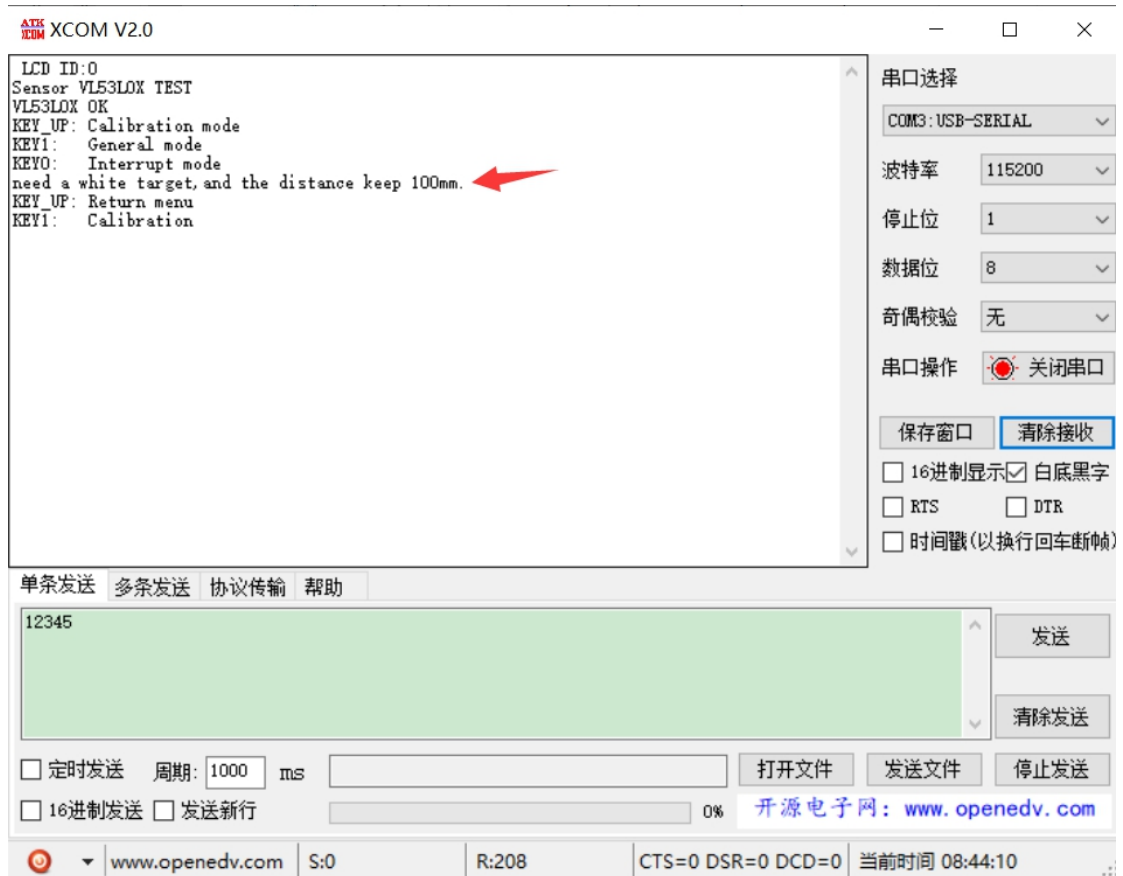
对于没有 TFTLCD 彩屏的用户，使用我们 PZ-VL53L0X 模块的时候，也可以使用串口来输出信息。串口输出如下图所示：



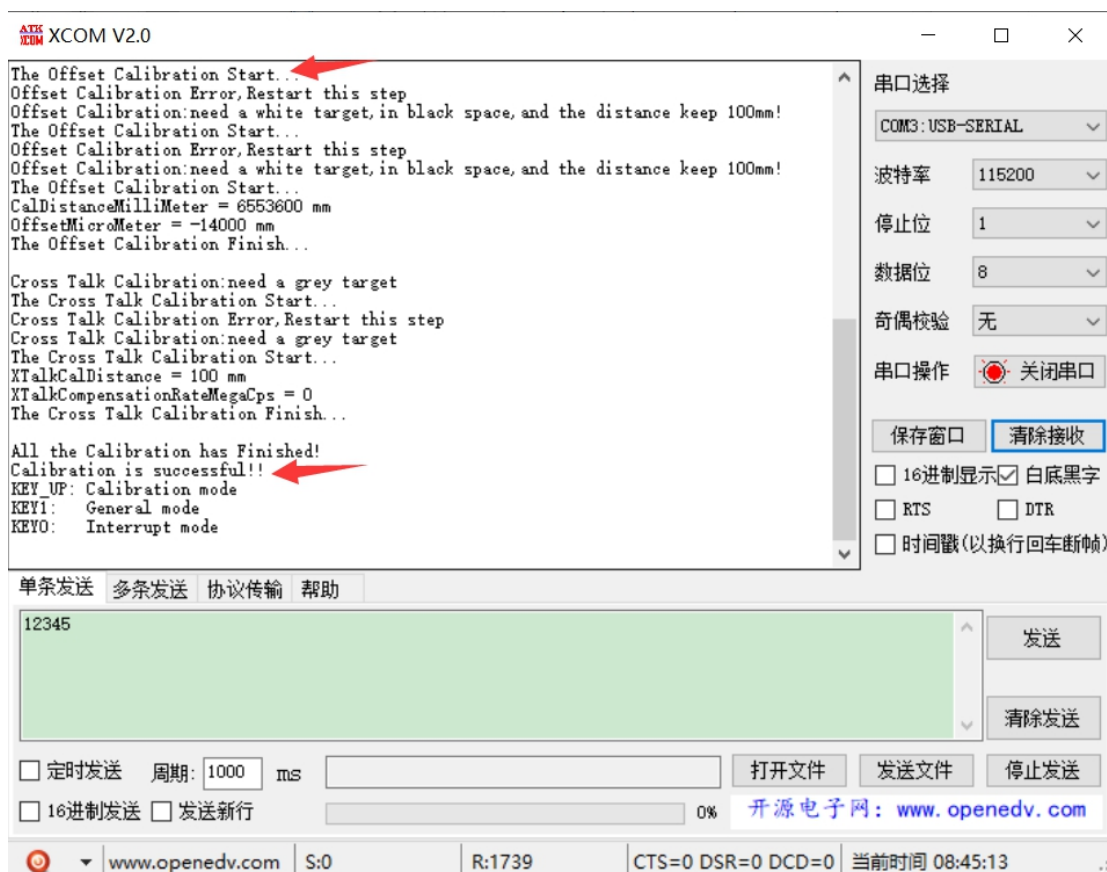
可以看到，串口助手显示 VL53L0X OK，表示 VL53L0X 传感器初始化通过，同时显示了 KEY_UP/KEY1/KEY0 测试功能选项，KEY_UP 校准测试、KEY1 普通测量测试、以及 KEY0 中断测量测试。

（1）校准测试

在主菜单界面，按 KEY_UP，则可进入此项测试，此项测试为校准 VL53L0X 传感器，校准测试主界面如下图所示：



串口助手显示校准需要一个白色目标物体，而且 VL53L0X 传感器与目标距离需保持 10 厘米（100mm）。当确认目标和距离无误后，按下 KEY1 执行校准。若不执行校准，则按下 KEY_UP 返回主菜单页面。按下 KEY1 按键后，这时会提示 Start Calibration，表示校准开始，

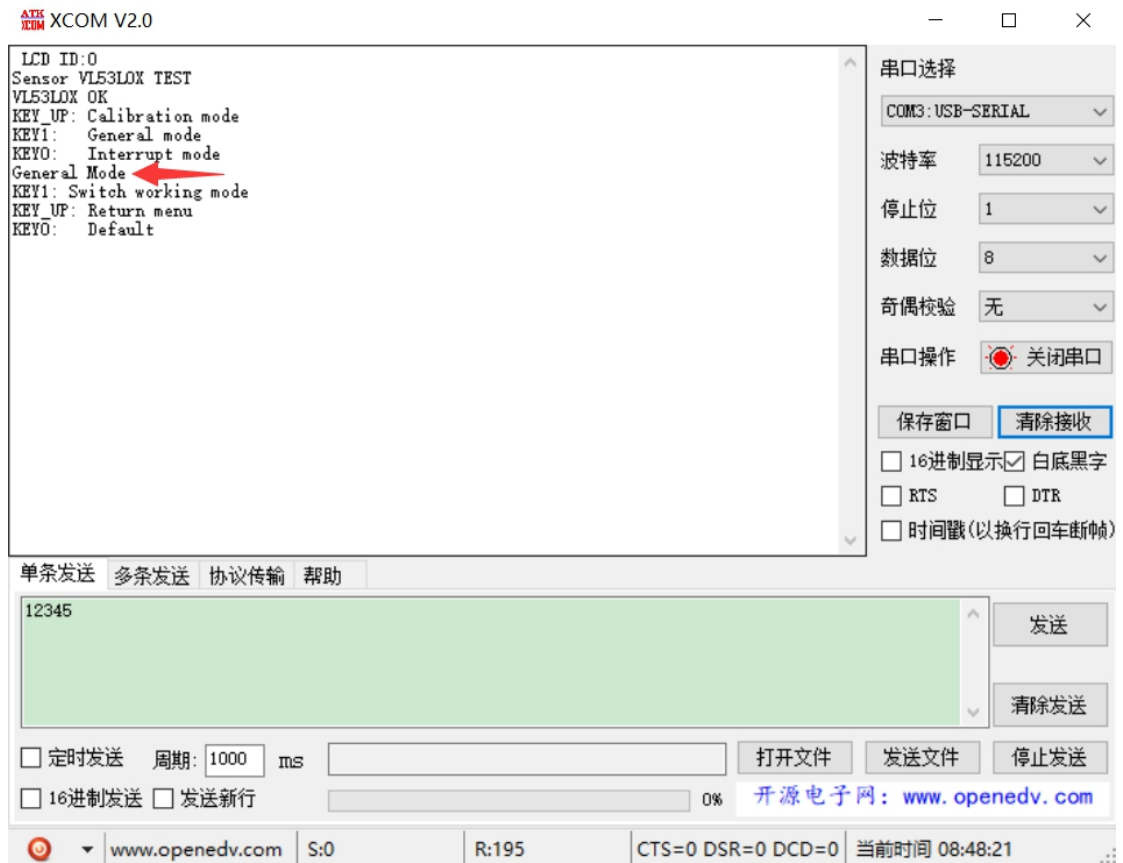


校准结束后，串口助手会显示校准状态，若显示 Calibration is successful!，则表示校准结束（成功），显示 Calibration is error，则表示校准失败，同时开发板 DS1 灯保持常亮以提示校准失败。（注意：校准失败有可能是当前 VL53L0X 传感器接收不到返回的激光信号，所以请检查模块与目标距离是否为 10 厘米）。

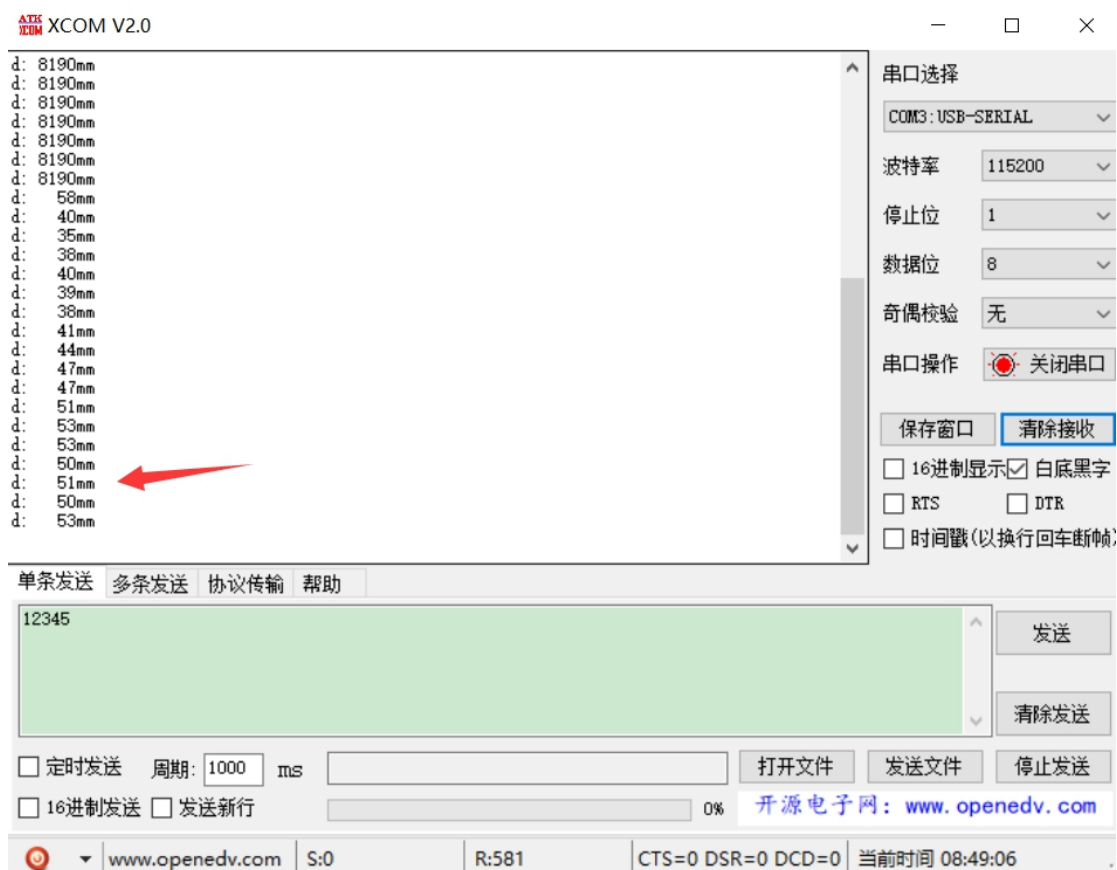
校准状态显示后，串口助手显示会自动返回主菜单页面。

（2）普通测量测试

在主菜单页面，按 KEY1，可进入此项测试，此项可测试在单次测量工作模式下，实现在不同精度模式下进行距离的测量。普通测量测试主界面如下图所示：



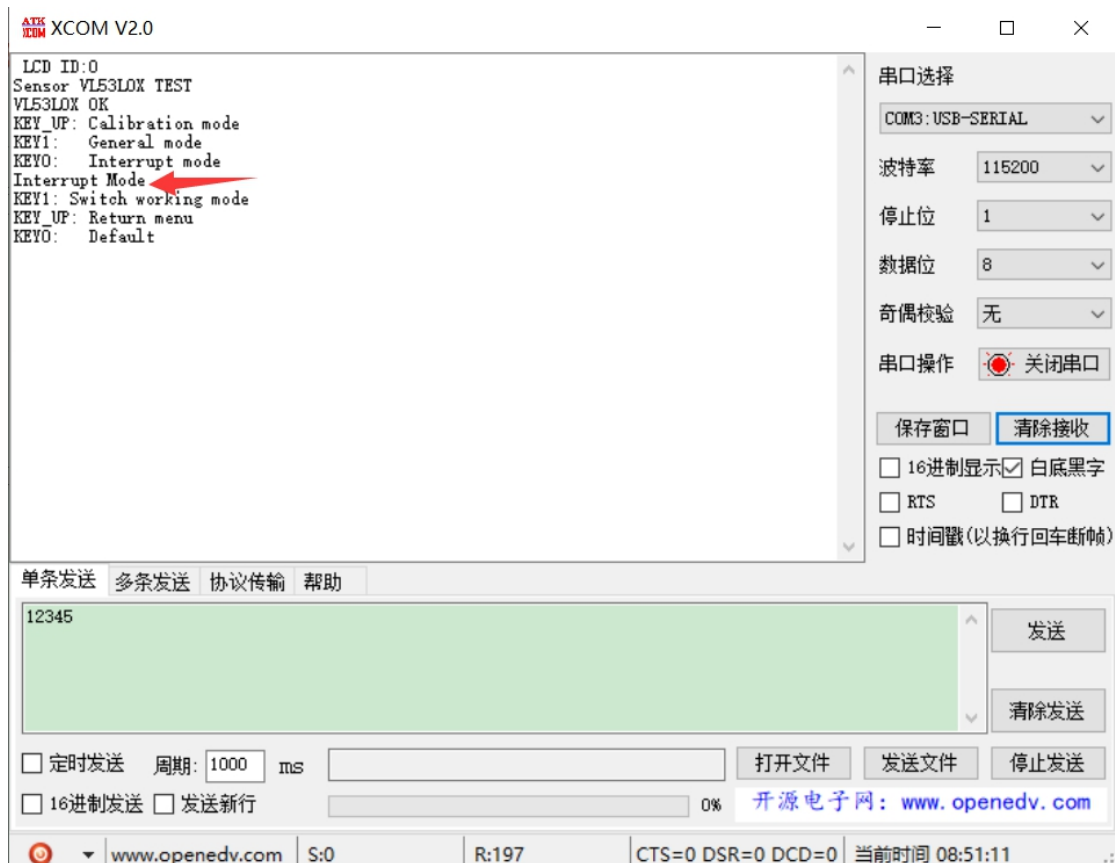
按 KEY1 可切换精度模式,从上图可以看到,当前设置为 Default 默认精度,
按 KEY0 进入测试,测试的界面如下图所示。



进入测试后，串口助手会显示当前测量的距离。单击 KEY_UP 会返回精度选择页面，双击 KEY_UP 则会返回主菜单页面。

(3) 中断测量测试

在主菜单页面，按 KEY0，可进入此项测试，此项可测试在连续测量工作模式下，利用中断触发，实现在不同精度模式下进行距离的测量。中断测量测试主界面如下图所示：



按 KEY1 可切换精度模式，从图中可以看到，当前设置的是 Default 默认精度，按 KEY0 可进入测试，测试的界面如下图所示。



进入测试后，串口助手会显示当前测量的距离。当测量的距离在上下限（默认上限为 150mm，下限为 60mm）距离范围内时，测量中断不触发，距离数据不做更新变化。当距离小于下限距离值或大于上限距离值时，测量中断触发，测量的距离数据发生变化。单击 KEY_UP 会返回精度选择页面，双击 KEY_UP 则返回主菜单页面。

至此，关于 PZ-VL53L0X 模块的使用介绍，我们就讲完了，本手册介绍了 PZ-VL53L0X 模块的使用，有助于大家快速学会 PZ-VL53L0X 模块的使用。

5 其他

(1) 购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

(2) 资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

(3) 技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）